사전 준비하기 구글 코랩으로 프로그래밍하기

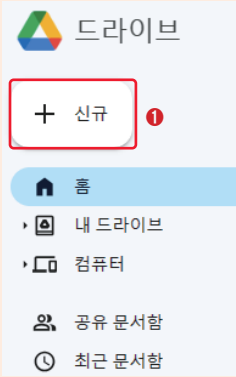
코랩 알아보기 (PDF)

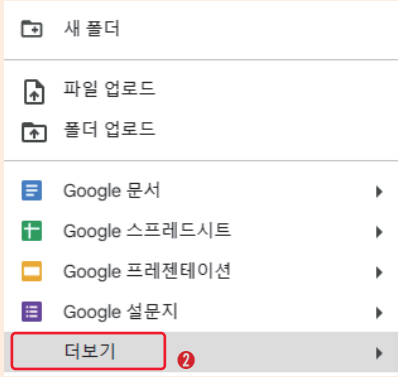
구글 코랩(Google Colab)은 온라인상에서 파이썬으로 프로그래밍하고 바로 결과를 확인할 수 있는 통합 개발 환경이다.

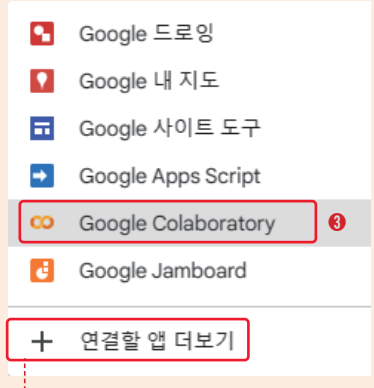
(1) 구글 코랩 파일 생성하기

① 크롬에서 구글 계정으로 로그인하고 (Google 앱) – [드라이브]를 선택한다.

② 드라이브 창에서 [신규] – [더 보기] – [Google Colaboratory]를 선택하여 코랩의 작업 창을 연다.





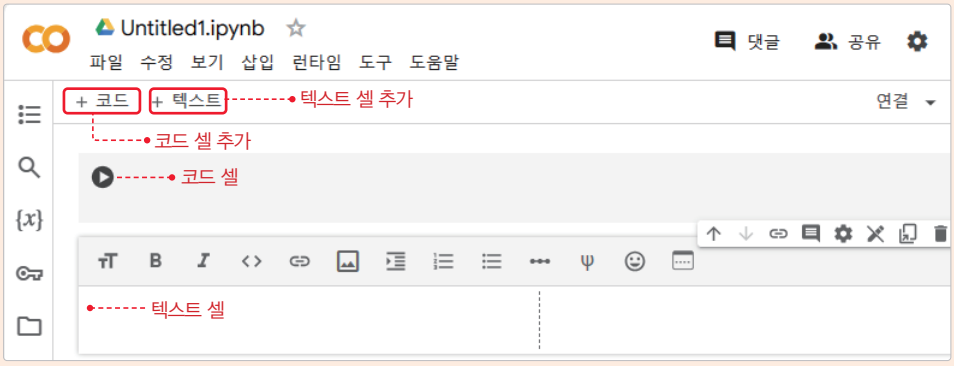


[Google Colaboratory]가 보이지 않은 경우 클릭

(2) 코랩의 작업 창 구성 알아보기

코랩은 크게 코드 셀과 텍스트 셀로 구성되어 있다.

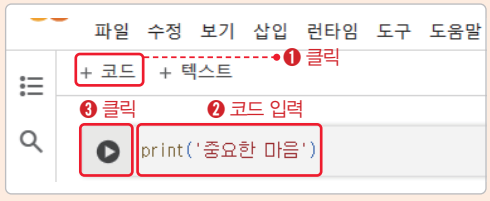
- 코드 셀: 코드를 작성하고, 실행시킨 결과를 확인할 수 있는 공간이다.
- 텍스트 셀: 글과 이미지를 작성하는 공간이다.

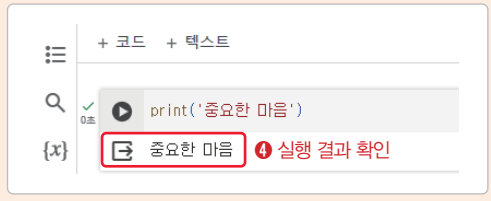


(3) 코랩에서 프로그램 실행하기

① 구글 코랩의 [새 노트] 창의 코드 셀에 코드를 작성한다.

② (실행) 버튼을 클릭하여 실행 결과를 확인한다.





1 변수와 자료형

우리가 중요한 일들을 잊지 않기 위해 메모를 하듯이 컴퓨터 프로그램은 중요한 데이터를 잊지 않도록 별도의 기억 공간에 값을 저장한다.

1 변수

변수는 프로그래밍에서 값을 저장하는 기억 공간이다. 등호(=)를 이용하여 변수에 값을 저장하고, 저장된 값을 변경할 수 있다.

▶형식 변수명 = 변수에 저장할 값(상수, 변수, 수식 등)

변수명은 기억 공간을 식별하기 위해 이름을 붙인 것으로 몇 가지 규칙만 지킨다면 자유롭게 설정할 수 있다.

변수명 지정 규칙
- 알파벳 대소문자, 한글, 언더 바(_), 숫자만을 사용할 수 있다.
- if, for 등과 같은 파이썬 예약어는 변수명으로 사용할 수 없다.
- 알파벳 대소문자는 구별해서 사용한다.
- 첫 글자에 숫자가 올 수 없다.

파이썬 예약어
파이썬에서는 다음과 같은 예약어를 변수명으로 사용할 수 없다.
예 True, False, and, or, not, return, for, while, if, else, elif, continue, break 등

예제 5 변수에 값을 저장하고 값을 변경해 보자.

```
01 # a에 정수 10을 저장하기
02 a = 10
03 # b, c에 실수 2.5와 문자 'A'를 순서대로 저장하기
04 b, c = 2.5, "A"
05 print(a, b, c)
06 # a에 저장된 값에 2를 곱하기
07 a = a * 2
08 print(a, b, c)
```

실행 결과
```
10 2.5 A
20 2.5 A
```

print( )
괄호 안의 값, 변수명 수식 결과를 출력한다.

활동 12 개발

다음은 이름, 나이, 연락처 정보를 저장하는 화면과 프로그램이다. 빈칸에 들어갈 값을 채워 보자.

이름 : 김민지
나이 : 17
연락처 : 010-1234-5678

```
01 name = '김민지'
02 age = [   ]
03 phone = [          ]
```

변수의 역할을 나타내는 변수명을 사용하면 프로그램 이해와 수정이 쉬워요.

2 자료형

자료형은 변수에 저장될 데이터가 어떤 유형인지 구분하는 것으로 종류에는 수치형, 불형, 문자열 등이 있다.

수치형은 숫자를 나타내는 자료형으로 정수형(int)과 실수형(float)이 있으며, 불형은 참(True)과 거짓(False)의 정보를 나타내는 자료형이다. 문자열은 문자를 나타내는 자료형으로 큰따옴표("")나 작은따옴표('')로 감싸서 표현한다.

예제 6 변수에 다양한 값을 저장하고 자료형을 출력해 보자.

```
01  a = 3
02  print(a, type(a))
03  b = 1.5
04  print(b, type(b))
05  c = True
06  print(c, type(c))
07  d = False
08  print(d, type(d))
09  e = "apple"
10  print(e, type(e))
11  f = "2025"
12  print(f, type(f))
```

실행 결과

```
3 <class 'int'>        ← 정수형
1.5 <class 'float'>    ← 실수형
True <class 'bool'>    ← 불형
False <class 'bool'>   ← 불형
apple <class 'str'>    ← 문자열
2025 <class 'str'>     ← 문자열
```

type( )
- 자료의 종류를 반환한다.
- 괄호 안에는 변수나 수식, 값이 들어갈 수 있다.

예제 7 문자열을 다양한 형태로 출력해 보자.

```
01  a = "HAPPY BIRTHDAY!"
02  print(a[1])      # 1번 인덱스 출력하기
03  print(a[6:11])   # 6~10번 인덱스 출력하기
04  print(a[ : 5])   # 0~4번 인덱스 출력하기
05  print(a[11: ])   # 11~문자열 끝 출력하기
06  b = "2U"
07  print(a + b)     # a와 b 문자열 결합하기
08  print(b * 2)     # b 문자열 반복하기
```

인덱스	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	H	A	P	P	Y		B	I	R	T	H	D	A	Y	!

인덱스	0	1
b	2	U

실행 결과

```
A
BIRTH
HAPPY
DAY!
HAPPY BIRTHDAY!2U
2U2U
```

인덱스(index)
저장된 값(요소)의 위치 데이터로 0부터 순서대로 1씩 증가한다. 문자열이나 리스트 자료형에서 사용한다.

개발 **문자열 a에서 HBD만 출력해 보자.**

3 연산자

프로그래밍에서 연산자는 특정한 작업을 수행하려고 사용하는 기호들로서 어떤 값을 조작하거나 식을 계산할 수 있다. 연산자의 종류에는 산술, 비교, 논리 연산자 등이 있고, 그 우선순위는 산술>비교>논리 연산자순이다.

산술 연산자의 우선순위
- 괄호 안의 연산식을 먼저 계산한다.
- 곱셈과 나눗셈은 덧셈과 뺄셈보다 우선순위가 높다.
- 덧셈과 뺄셈, 곱셈과 나눗셈 등 연산자 우선순위가 같을 땐 왼쪽부터 오른쪽으로 계산한다.

(1) 산술 연산자

수치형 데이터들을 대상으로 연산을 수행하는 연산자로 +(덧셈), −(뺄셈), *(곱셈), /(나눗셈), //(몫), %(나머지), **(거듭제곱) 등이 있다.

예제 8 산술 연산자를 이용하여 간단한 수식을 계산해 보자.

```
01 print(11 + 4 / 2)
02 print(25 // 2)
03 print(25 % 2)
04 print(2 ** 10)
```

실행 결과
```
13.0
12
1
1024
```

(2) 비교 연산자

두 개 이상의 값이나 변수에 저장된 값들을 비교하여 True(참) 또는 False(거짓)를 판단하는 연산자이다. 연산자로는 >(크다.), <(작다.), >=(크거나 같다.), <=(작거나 같다.), ==(같다.), !=(같지 않다.) 등이 있다.

예제 9 비교 연산자를 이용하여 값의 크기를 비교해 보자.

```
01 n = 10                # 변수 n에 10 저장하기
02 print(n == 5)          # n값이 5와 서로 같은지 비교하기
03 print(n != 5)          # n값이 5와 서로 다른지 비교하기
04 print(n > 5)           # n값이 5보다 큰지 비교하기
05 print(n <= 5)          # n값이 5보다 작거나 같은지 비교하기
06 print(5 <= n < 10)     # n값이 5 이상 10 미만인지 비교하기
```

실행 결과
```
False
True
True
False
False
```

활동 13 개발

1 키(m)와 몸무게(kg)를 변수에 저장하고 체질량 지수를 계산하는 프로그램을 완성해 보자.

$$\text{체질량 지수(bmi)} = \frac{\text{몸무게(kg)}}{\text{키의 제곱(m)}^2}$$

```
01 h = 1.7          # 키
02 w = 60           # 몸무게
03 bmi = [          ]
04 print(bmi)
```

2 1번에서 구한 체질량 지수(bmi)가 정상 범위에 속하는지 그 결과를 True나 False로 출력하는 명령문을 완성해 보자. 단, 정상 범위는 체질량 지수가 18.5 이상 23 미만인 경우이다.

```
print ( [                ] )
```

(3) 논리 연산자

논리식의 참과 거짓을 판단하는 연산자로 논리곱(and), 논리합(or), 논리 부정(not) 등이 있다.

논리 연산자의 종류
- 논리곱(and)
  예 a and b: a와 b 모두가 참일 때 결괏값이 참이 된다.
- 논리합(or)
  예 a or b: a 또는 b 중 하나 이상이 참이면 결괏값도 참이 된다.
- 논리 부정(not)
  예 not a: a값이 참이면 결괏값은 거짓, a값이 거짓이면 결괏값은 참이 된다.

예제 10 논리곱(and), 논리합(or) 연산자를 사용하여 참, 거짓을 판단해 보자.

```
01 id = "Lee"
02 pw = "1234"
03 print(id == "Lee" and pw == "1234")
04 print(id == "Kim" and pw == "1234")
05 print(id == "Lee" or pw == "asdf")
06 print(id == "Kim" or pw == "asdf")
```

실행 결과
```
True
False
True
False
```

예제 11 논리 부정(not) 연산자를 사용하여 식의 결과를 확인해 보자.

```
01 a = 10 > 5
02 print(a)
03 print(not a)
```

실행 결과
```
True
False
```

활동 14 개발

[혈압 단계 구분] 표와 [실행 결과]를 참고하여 고혈압 단계를 알려 주는 프로그램을 완성해 보자.

▼ 혈압 단계 구분

혈압 단계	수축기 혈압	연산자	이완기 혈압
정상 혈압	120 미만	그리고	80 미만
주의 혈압	120 ~ 129	그리고	80 미만
고혈압 전 단계	130 ~ 139	또는	80 ~ 89
고혈압	140 이상	또는	90 이상

실행 결과
```
수축기 혈압 135 이완기 혈압 88
정상 혈압: False
주의 혈압: False
고혈압 전 단계: True
고혈압: False
```

```
01 a, b = 135, 88                                   # a는 수축기 혈압, b는 이완기 혈압
02 print("수축기 혈압", a, "이완기 혈압", b)
03 print("정상 혈압:", a < 120 and b < 80 )
04 print("주의 혈압:", [                    ] )
05 print("고혈압 전 단계:", [                    ] )
06 print("고혈압:", [                    ] )
```